

Sistema de Riego Automatizado

Ruben Chirino

24 Noviembre, 2023



Índice

Caratula	0
Índice	1
Consignas	2
Introducción	4
Objetivos general y específicos	5
Diagrama de Gantt	6
Diagrama por bloques	7
Desarrollo y Diseño de Hardware y Software	9
Ensayos y mediciones	14
Conclusiones	17
Bibliografía	19
Anexo	20

Consignas

1. **Prototipo:** desarrollar un sistema prototipo de hardware y software que utilice al menos **2 sensores y 2 actuadores** utilizando las tecnologías vistas en clase tales como NodeMCU, Arduino, y/o similares.

Ejemplo: La realización de una estación meteorológica en donde se tenga mínimo de dos sensores como pueden ser de humedad, y otro de temperatura y como actuadores una bomba de agua y un display del tipo LCD.

El sistema a desarrollar deberá ser un prototipo de lo que se desea hacer y que deberá ser presentado en formato físico en las instancias de defensa (presencial y/o virtuales) del TP.

2. **Informe final:** se deberá desarrollar un informe en el cual consten lo siguientes puntos:
 - a. **Caratula:** acá deberán colocar datos de los integrantes, nombre de profesor, materia, año y cuatrimestre y nombre del trabajo. En cuanto al nombre del trabajo se espera que se pueda entender fácilmente qué se desea realizar, es decir que sea descriptivo dentro de lo posible.
 - b. **Índice:** cada una de las secciones que se desarrollarán en el trabajo deberán estar en el índice con su número de página. A continuación se deja una sugerencia de índice a realizar.
 - c. **Introducción:** en esta sección se deberá desarrollar brevemente de qué va a tratar el trabajo, contar qué se pretende hacer y cómo, con qué tecnologías, que se desea lograr.
 - d. **Objetivos general y específicos:** en esta sección se debe desarrollar el objetivo general -primario y los objetivos específicos o secundarios.
 - e. **Diagrama de Gantt:** realizar el diagrama de Gantt que da cuenta de los tiempos de desarrollo del proyecto.
 - f. **Diagrama por bloques:** deberán realizar un diagrama por bloques o de forma gráfica que dé cuenta de cada parte involucrada en el sistema y cómo se relacionan estas partes entre sí en el sistema total. (ver punto 2).



- g. Desarrollo y Diseño de Hardware y Software:** en estas secciones deberá desarrollar todo lo relativo al hardware y software que interviene en el sistema, listar los componentes, sus precios, justificar su elección por sobre otros componentes y describir el funcionamiento.
- h. Ensayos y mediciones:** deberán dar cuenta aquí de los ensayos realizados, las pruebas realizadas, mediciones, recolección de datos, y todo lo realizado para comprobar que el sistema responde a los requerimientos.
- i. Conclusiones:** dar cuenta de las principales conclusiones a las que se arribó en el proyecto desde su inicio hasta el final, mostrando los aspectos más relevantes del mismo.
- j. Bibliografía:** acá deberán colocar los links, libros y material de consulta para la realización del trabajo. Deberá cumplir con citas APA.
- k. Anexo:** en esta sección se colocará toda imagen y material complementario al trabajo principal realizado. Por ejemplo la realización de un procedimiento específico para trabajar.



Introducción

En la era actual, donde la tecnología se entrelaza cada vez más con todos los aspectos de nuestras vidas, nuestro proyecto **"Harvest Guardian"** emerge como una solución innovadora en el campo de la jardinería y el cuidado de las plantas. Este proyecto representa una integración excelente de hardware y software, destinado a simplificar y optimizar el proceso de riego y cuidado de plantas, aprovechando la tecnología de automatización y control remoto.

"Harvest Guardian" es un Sistema de Riego Automatizado que utiliza un conjunto de hardware, incluyendo un microcontrolador ESP32, sensores de luz (LDR) y humedad del suelo, una mini bomba de agua, un protoboard con relé, manguera y un reservorio de agua. Todo esto es alimentado por una batería de 9V, lo que garantiza una fuente de energía confiable y constante. Este sistema está diseñado para ofrecer una solución integral para el riego automático y el monitoreo ambiental de las plantas.

No es solo una herramienta de riego, sino una plataforma completa de gestión del cuidado de las plantas que representa un paso adelante en la jardinería moderna. Combina tecnología de vanguardia con una interfaz de usuario amigable, lo que lo hace accesible para jardineros de todos los niveles de habilidad, desde aficionados hasta expertos.

Objetivos

General

Desarrollar e implementar un sistema de riego automatizado y monitorización ambiental para plantas, integrado con una interfaz de usuario basada en un Bot de Telegram, con el fin de optimizar el cuidado de las plantas y facilitar la jardinería para usuarios de todos los niveles.

Específicos

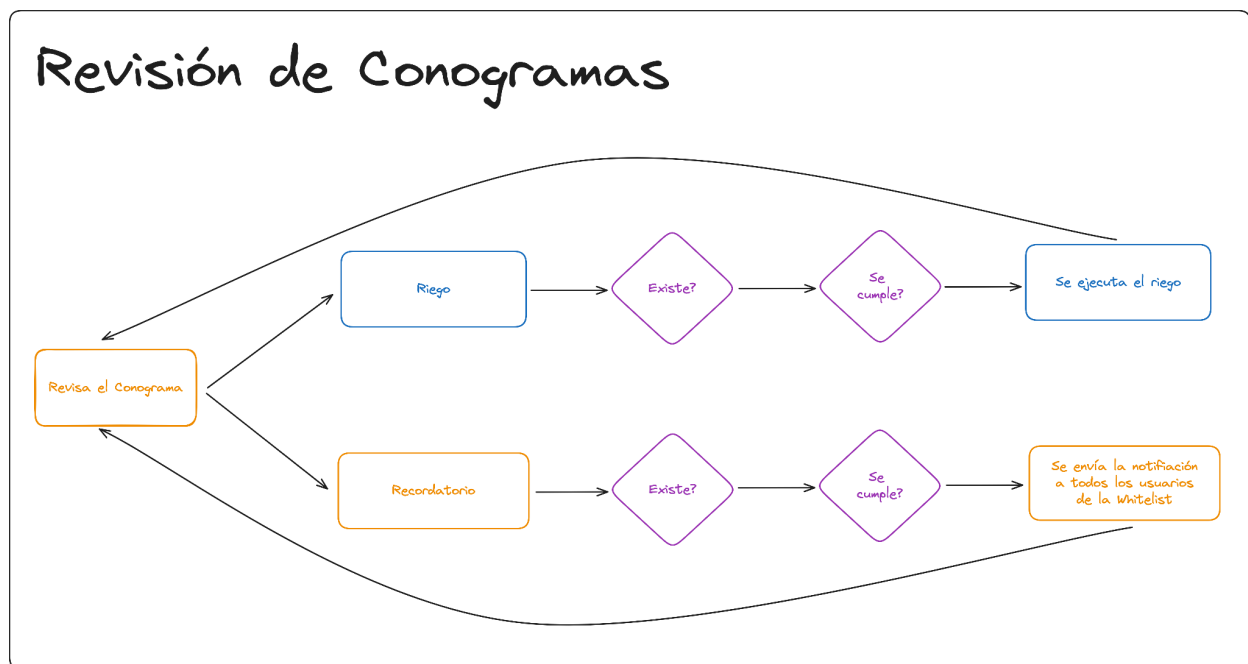
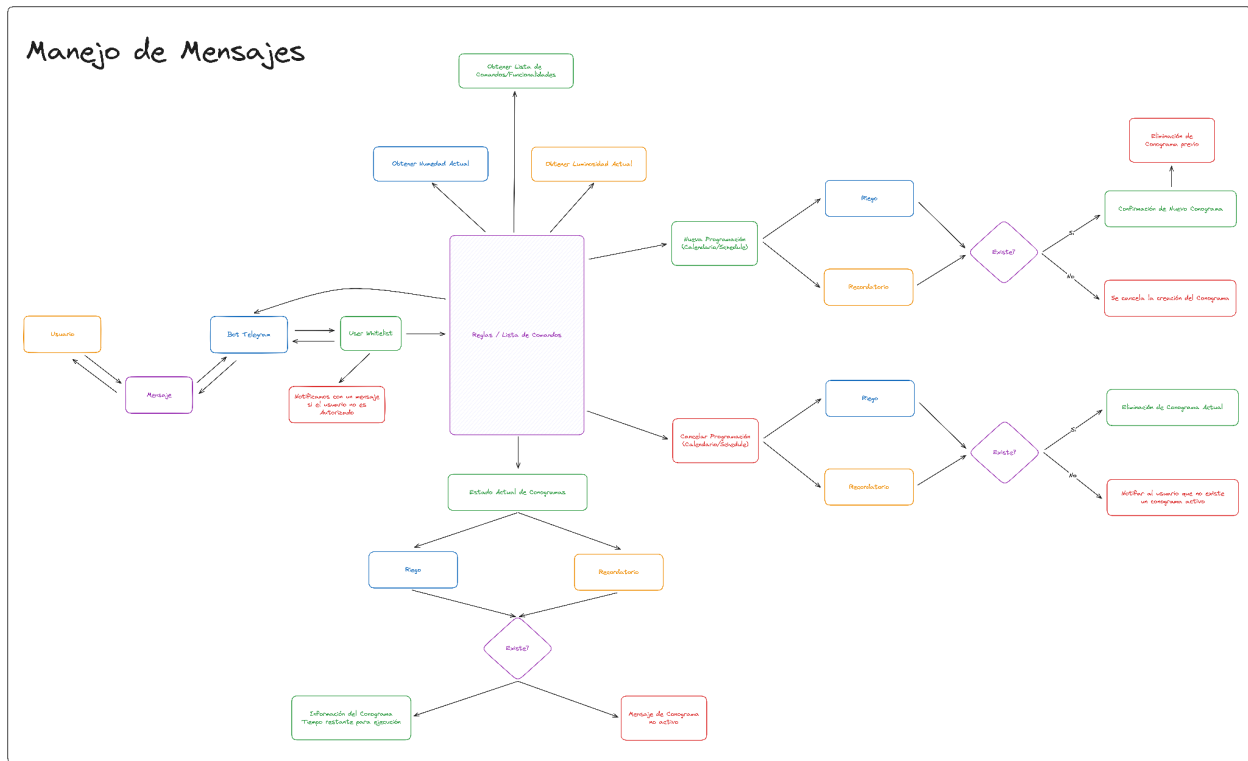
1. **Integración de Hardware y Software:** Combinar eficientemente componentes de hardware como el ESP32, sensores de humedad del suelo y luz (LDR), una mini bomba de agua y un sistema de relés, con un software que permita su control y monitoreo remoto.
2. **Automatización del Riego:** Configurar un sistema que permita a los usuarios establecer horarios de riego recurrentes y específicos, adaptándose a las necesidades hídricas particulares de distintos tipos de plantas y usuarios.
3. **Monitorización Ambiental:** Implementar sensores para monitorear constantemente factores ambientales clave como la humedad del suelo y el nivel de luz, proporcionando datos precisos para un cuidado óptimo de las plantas.
4. **Desarrollo de una Interfaz de Usuario Amigable:** Crear un Bot de Telegram que sirva como interfaz principal para interactuar con el sistema, ofreciendo una experiencia de usuario sencilla, intuitiva y accesible para personas con variados niveles de experiencia en tecnología.
5. **Mejora de la Eficiencia en el Cuidado de Plantas:** Reducir la necesidad de intervención manual en el riego y cuidado de las plantas, a través de la automatización y el suministro de información ambiental relevante en tiempo real.
6. **Promoción de la Jardinería Sostenible:** Contribuir a prácticas de jardinería más sostenibles y eficientes en el uso del agua, a través de la automatización precisa del riego y el monitoreo del entorno de las plantas.

Diagrama de Gantt

DIAGRAMA DE GANTT Calendario de tareas del proyecto						
TAREAS	SEMANA 1	SEMANA 2	SEMANA 3	SEMANA 4	SEMANA 5	SEMANA 6
TAREA 1	Definición de Requisitos	Diseño de la Arquitectura del Sistema	Desarrollo de la Lógica Básica	Creación y conexión del Bot	Montaje de componentes restantes	Pruebas Finales
TAREA 2	Investigación	Diseño Preliminar del Software y la Interfaz	Montaje de algunos componentes	Desarrollo de Funciones del Bot	Desarrollo de la Comunicación (Parte 2)	Desarrollo del Informe
TAREA 3	Selección de Componentes	Adquisición de Componentes	Desarrollo de la Comunicación (Parte 1)	Pruebas de la lógica	Integración de Hardware y Software	Documentación
TAREA 4	Selección de Herramientas	Configuración del SetUp de desarrollo		Resolución de Bugs	Pruebas del sistema	
TAREA 5					Resolución de Bugs	

[Link al archivo PDF del Diagrama de Gantt](#)

Diagrama de bloques del Funcionamiento



Explicación Complementaria de los Diagramas

Usuario: Interactúa con el Bot de Telegram mediante **mensajes** para realizar acciones o solicitar información.

Bot de Telegram: Recibe **mensajes** del usuario.

Mensajes: Son instrucciones/comandos los cuales son validados por las reglas definidas.

Lista de Reglas: Es la lista de comandos aceptados por el Bot (Todas las instrucciones que el Bot devuelve con el comando `"/start"`).

Funcionalidades: Tenemos 2 tipos, las de **acción** y/o **reporte**.

- Acción: Son aquellas que se encargan de realizar una operación que involucra una ejecución física ya sea en el momento de la ejecución o en el futuro, por ejemplo, establecer un nuevo cronograma (`"/set_watering_schedule"` o `"/set_reminder_schedule"`).
- Reporte: Son aquellas que se encargan de hacer una lectura del estado del sistema ó una devolución de información, solicitar la lista de comandos (`"/start"`), el estado de los cronogramas (`"/schedules_state"`) y obtener el porcentaje de luz (`"/light_level"`) o humedad (`"/soil_moisture"`) son ejemplos de esto.

[Link al Scalidrow con las Imágenes de los Diagramas presentados.](#)

[Imagen del Diagrama de Manejo de Mensajes](#)

[Imagen del Diagrama de Revisión de Cronogramas](#)

[Imagen del Diagrama de Conexiones de Componentes](#)

Desarrollo y Diseño de Hardware

Componentes Utilizados:

1. ESP32:

- **Función:** Núcleo de control y comunicación.
- **Justificación:** El ESP32 fue elegido por su capacidad Wi-Fi integrada, procesamiento potente y soporte para múltiples entradas/salidas. Es ideal para proyectos IoT.
- **Alternativas Consideradas:** Arduino Uno, Raspberry Pi. El ESP32 fue preferido por su conectividad nativa y mayor capacidad de procesamiento en comparación con Arduino, y su menor costo y tamaño en comparación con Raspberry Pi.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

2. Sensor de Humedad del Suelo:

- **Función:** Medición de la humedad del suelo.
- **Justificación:** Esencial para automatizar el riego basado en las necesidades reales de las plantas.
- **Alternativas Consideradas:** Sensores de humedad más avanzados. Se eligió este por su relación costo-eficiencia y facilidad de integración.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

3. Sensor de Luz (LDR):

- **Función:** Medir el nivel de luz.
- **Justificación:** Permite ajustar las rutinas de riego según la cantidad de luz solar, lo que es útil para la gestión del riego.
- **Alternativas Consideradas:** Sensores de luz digitales. Se eligió el LDR por su simplicidad y costo.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

4. Mini Bomba de Agua:

- **Función:** Realizar el riego.
- **Justificación:** Componente clave para el mecanismo de riego, seleccionada por su tamaño y eficiencia.
- **Alternativas Consideradas:** Bombas más grandes, pero se optó por esta por su adecuación a pequeñas instalaciones de jardinería.

- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

5. Relé:

- **Función:** Actuar como interruptor para la bomba.
- **Justificación:** Necesario para controlar la bomba de agua de mayor potencia que lo que el ESP32 puede manejar directamente.
- **Alternativas Consideradas:** Transistores para cargas menores, pero el relé fue necesario para manejar la corriente de la bomba.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

6. Batería de 9V (y adaptador):

- **Función:** Fuente de alimentación.
- **Justificación:** Proporciona una fuente de alimentación portátil y fácil de reemplazar.
- **Alternativas Consideradas:** Fuentes de alimentación USB. La batería de 9V fue elegida por su simplicidad y portabilidad.
- **Links:**
 - [Batería 9V](#) (Acá se encuentra el precio actualizado).
 - [Adaptador](#) (Acá se encuentra el precio actualizado).

7. Protoboard:

- **Función:** Facilitar las conexiones de los componentes.
- **Justificación:** Permite una fácil experimentación y modificación del circuito.
- **Alternativas Consideradas:** PCB personalizado, pero se optó por un protoboard por su flexibilidad y coste.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

8. Resistencias:

- **Función:** Limitar la corriente y proteger componentes sensibles, como el LDR.
- **Justificación:** Las resistencias son esenciales para el correcto funcionamiento de circuitos con sensores y para proteger el ESP32 de corrientes excesivas.
- **Alternativas Consideradas:** No hay alternativas directas a las resistencias para sus funciones específicas.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

9. Cables para Conexiones:

- **Función:** Conectar físicamente los componentes en el protoboard y al ESP32.
- **Justificación:** Los cables son necesarios para establecer conexiones eléctricas entre los componentes. Se prefieren los cables jumper por su flexibilidad y facilidad de uso.
- **Alternativas Consideradas:** Cables sólidos o prensados, pero los jumpers son más convenientes para prototipos y ajustes.
- **Links:**
 - [Cables Macho - Hembra](#) (Acá se encuentra el precio actualizado).
 - [Cables Macho - Macho](#) (Acá se encuentra el precio actualizado).

10. Cable USB:

- **Función:** Conectar el ESP32 a una fuente de alimentación o a un computador para programación.
- **Justificación:** El cable USB es esencial para programar el ESP32 y también puede ser utilizado para alimentar el dispositivo durante las pruebas.
- **Alternativas Consideradas:** Fuentes de alimentación dedicadas, pero el USB es más conveniente y universal.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

11. Manguera de Silicona:

- **Función:** Conectar la mini bomba de agua al reservorio de agua, facilitando el transporte del agua hacia las plantas.
- **Justificación:** La manguera de silicona es esencial para dirigir el flujo de agua desde la bomba hasta las plantas. Se elige silicona por su flexibilidad, durabilidad y resistencia a la degradación por el agua.
- **Alternativas Consideradas:** mangueras de PVC o plástico, pero la silicona ofrece una mayor flexibilidad y es más fácil de manejar en configuraciones de jardinería doméstica.
- [Link al Producto Comprado](#) (Acá se encuentra el precio actualizado).

Desarrollo y Diseño de Software

Componentes Utilizados:

1. **Arduino IDE:** Utilizado para programar el ESP32, ofrece una interfaz amigable para escribir, compilar y cargar el código al microcontrolador. Además, proporciona acceso a una amplia gama de bibliotecas y ejemplos que facilitan el desarrollo de software para dispositivos IoT.
2. **Board:** Se requirió agregar/utilizar el siguiente link en la sección de "**Boards Adicionales**" en el apartado "**Preferencias**" del **Arduino IDE** para cargar los board relacionados al **ESP32**.

Link: https://dl.espressif.com/dl/package_esp32_index.json

Una vez descargado, en el apartado **Herramientas > Boards > esp32**, se deberá seleccionar nuestro modelo específico, el cual es el **NodeMCU-32s**.

3. **Librerías Utilizadas:**

- **WiFi y WiFiClientSecure:** Para conectarse a la red Wi-Fi.
- **NTPClient y WiFiUdp:** Para obtener la hora actual a través de un servidor NTP.
- **UniversalTelegramBot:** Para integrar el bot de Telegram.
- **ArduinoJson:** Para el manejo de datos en formato JSON.

4. **Funciones Principales:**

- **setup():** Inicializa los pines y se conecta a la red WiFi.
- **loop():** Función principal que verifica constantemente los mensajes del bot y los disparadores de riego.
- **checkNewMessages():** Procesa los nuevos mensajes recibidos en el bot de Telegram y ejecuta comandos.
- **checkSchedule():** Verifica si un horario específico debe ser ejecutado en el momento actual.

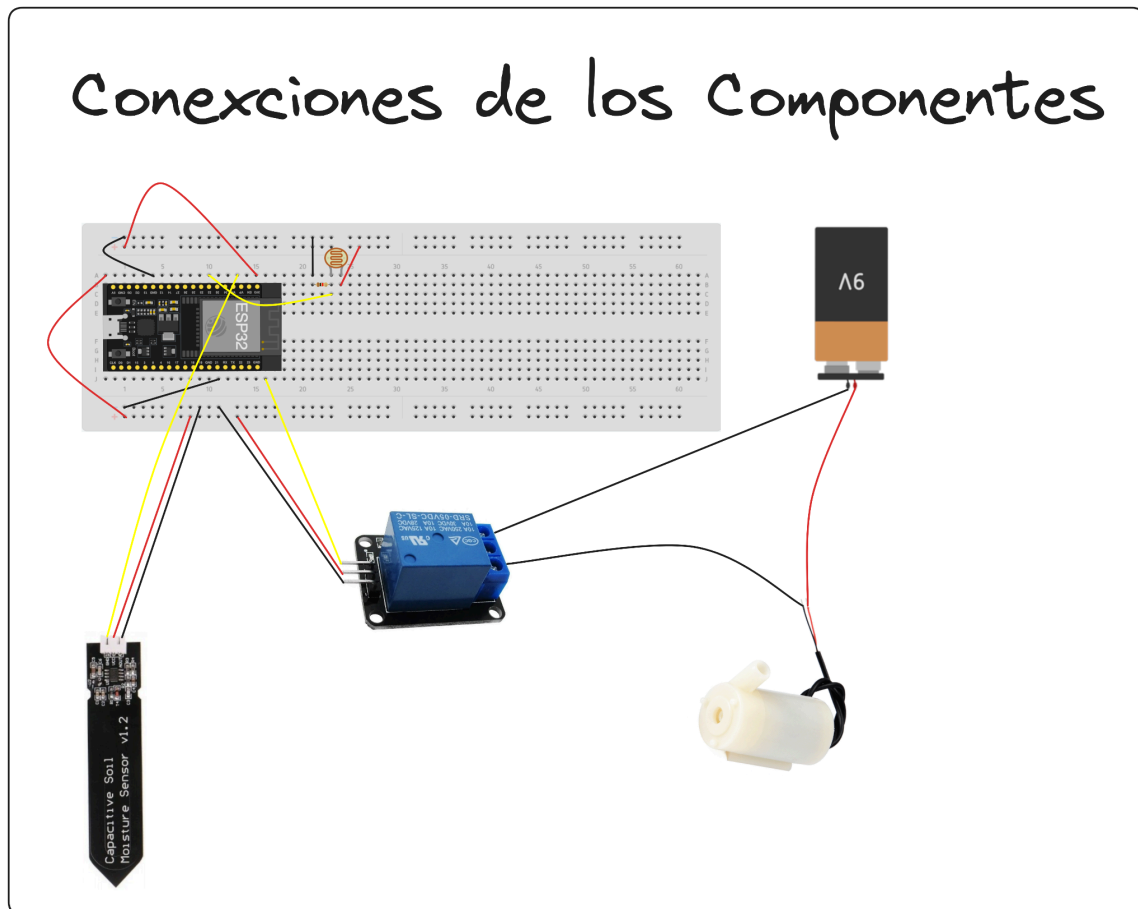
5. **Interfaz de Usuario a través de Telegram:** Debido a que estamos conectando nuestro ESP32 con un Bot de Telegram contamos con la interfaz de usuario de dicha aplicación, facilitando la comunicación para configurar horarios, recibir recordatorios y obtener lecturas de sensores.

Consideraciones de Diseño

- **Modularidad y Escalabilidad:** El código está estructurado de manera modular, permitiendo fácil mantenimiento y posibles ampliaciones futuras.
- **Comentarios y Documentación en Código:** Cada función y sección importante del código está documentada para facilitar su comprensión y mantenimiento.
- **Registro de Actividades:** Uso intensivo de `Serial.print()` para el registro y depuración durante el desarrollo.

El software representa un balance entre la complejidad técnica y la accesibilidad para el usuario, proporcionando una solución integral y automatizada para el riego de plantas.

[Link al Código en el repositorio de GitHub](#)



Ensayos y mediciones

Para esta sección del informe, se consideran tanto los ensayos realizados como las pruebas efectuadas para garantizar el correcto funcionamiento del sistema. Dado que el proyecto es un prototipo que incorpora varios sensores y actuadores, se han llevado a cabo diversas pruebas para validar cada componente y su interacción.

Puesto que no se pueden adjuntar videos en un documento, se han insertados los links a los videos de las pruebas alojados en Google Drive.

Interacción con el Bot

- [Usuario No Autorizado](#)
- [Usuario Autorizado](#)
- [Comando no existente](#)

Obtener Registros

- [Nivel de luz](#)
- [Nivel de humedad del suelo](#)
- [Estado de Horarios / Cronogramas](#) (Sin Horarios activos)

Intervalos de Riego Recurrentes

- [Nuevo riego recurrente](#) (Validación de rango de tiempo demasiado corto entre riegos)
- [Nuevo riego recurrente](#)
- [Cancela un intervalo de riego recurrente activo](#)
- [Cancela un intervalo de riego recurrente](#) (Validación, no existe ningún riego recurrente activo)

Horarios / Cronograma de Riego

- [Nuevo horario de riego](#)
- [Cancela un horario de Riego](#)
- [Cancela un horario de Riego](#) (Validación, no existe ningún horario de riego activo)
- [Revisa los horarios activos de riego y el tiempo restante](#)

Horarios / Cronograma de Recordatorios

- [Nuevo horario de Recordatorio](#)
- [Cancela un horario de Recordatorio](#)
- [Cancela un horario de Recordatorio](#) (Validación, no existe ningún horario de recordatorio activo)
- [Revisa los horarios activos de Recordatorio y el tiempo restante](#)

Ejecución de Horarios / Cronogramas

- Horario de riego cumplido (Se realiza el riego) **PENDING** ❌
- [Horario de recordatorio cumplido](#) (Se envía el recordatorio a todos los usuarios de la whitelist) (En el video se muestra que el recordatorio le llega a dos usuarios distintos)

En cada una de las pruebas anteriores se han probado y demostrado el funcionamiento de los comandos del Bot, los cuales se listan a continuación:

- **/set_recurring_watering_interval** [horas] - Establece el intervalo de riego automático. Por ejemplo, `/set_recurring_watering_interval 5` empezará a regar cada cinco horas.
- **/stop_recurring_watering_interval** - Detiene el sistema de riego automático.
- **/set_watering_schedule** [Days] [Time] - Establece un programa de riego específico. Por ejemplo, `set_watering_schedule Monday 18:00` establecerá un horario de riego los Lunes a las 6:00 PM.
- **/stop_watering_schedule** - Cancela el programa de riego actual.
- **/set_reminder_schedule** [Days] [Time] [Text] - Establece un horario de recordatorio específico. Por ejemplo, `/set_reminder_schedule Monday 18:00 Regar!` le recordará que debe "Regar!" las plantas a las 6:00 PM los Lunes.
- **/stop_reminder_schedule** - Cancela la programación de recordatorio actual.
- **/schedules_state** - Obtiene los horarios activos y los tiempos restantes para su ejecución.
- **/soil_moisture** - Obtiene el nivel actual de humedad del suelo.
- **/light_level** - Informa el nivel de luz actual detectado por el sensor LDR.



Los ensayos realizados tuvieron como objetivo principal validar la funcionalidad y fiabilidad del sistema prototipo de riego. Se comprobó la correcta operación de los sensores, actuadores y la integración con el software desarrollado.

Conclusiones de los Ensayos

Los ensayos demostraron que nuestro prototipo de riego automatizado cumple con los requerimientos y es capaz de adaptarse a diferentes condiciones, proporcionando un riego efectivo y eficiente. La integración del hardware con el software a través del bot de Telegram permite un control remoto y flexible del sistema de riego.

Conclusiones

Síntesis del Proyecto

El proyecto representa un avance en la gestión y automatización del cuidado de plantas. A través de la integración de hardware avanzado, como el ESP32, y sensores de humedad del suelo y luz, junto con una interfaz de usuario accesible vía Telegram, se ha desarrollado un sistema eficiente y fácil de usar.

Desafíos y Aprendizajes

- **Integración de Software y Hardware:** Uno de los principales desafíos fue la integración efectiva del software con el hardware. Asegurar la comunicación fluida y precisa entre el ESP32, los sensores y los actuadores requirió un entendimiento profundo de la programación de microcontroladores y la gestión de señales digitales y analógicas.
- **Desarrollo del Bot de Telegram:** La creación y configuración del bot de Telegram fue un área clave de aprendizaje. Se adquirieron habilidades en la programación de bots, especialmente en la implementación de comandos específicos y la gestión de interacciones con el usuario. Esto incluyó el manejo de autorizaciones y la interpretación de comandos para realizar acciones específicas en el sistema de riego.
- **Manejo de Horarios y Cronogramas:** Otro desafío importante fue el manejo de los horarios y cronogramas de riego. Se aprendió a implementar funciones para calcular y mostrar el tiempo restante hasta el próximo riego, basándose en la hora y fecha actual, y los horarios programados. Este aspecto fue crítico para garantizar que el sistema pudiera operar de manera autónoma y eficiente, adaptándose a las necesidades programadas y a las condiciones cambiantes.



Logros Alcanzados

1. **Automatización Eficiente:** Se ha conseguido automatizar el proceso de riego, adaptándolo a las necesidades específicas del suelo y las condiciones ambientales, lo que garantiza un uso más eficiente del agua.
2. **Interfaz de Usuario Intuitiva:** El uso de un bot de Telegram como interfaz ha hecho que el sistema sea accesible y manejable desde cualquier lugar, proporcionando a los usuarios un control remoto y en tiempo real.
3. **Flexibilidad y Personalización:** La capacidad de programar horarios de riego y recibir datos en tiempo real sobre las condiciones del suelo y la luz permite a los usuarios personalizar el riego según las necesidades específicas de sus plantas.

Conclusión Final

El proyecto demuestra con éxito la viabilidad y los beneficios de un sistema de riego automatizado controlado a través de un bot de Telegram. Representa un paso adelante en la combinación de tecnología y jardinería, ofreciendo una solución práctica y sostenible para el cuidado de plantas. Con su potencial de expansión y adaptación, abre un camino innovador hacia una gestión más inteligente y eficiente del agua en la agricultura y jardinería.

Bibliografía

En el desarrollo de este proyecto, se consultaron diversas fuentes de información que abarcan desde manuales técnicos hasta tutoriales y guías en línea. A continuación, se presenta una lista de las referencias más importantes:

1. [Arduino Reference](#) (Lista y documentación de las funciones para controlar la placa Arduino y realizar cálculos).
2. [Telegram Bot API](#)
3. [Universal Telegram Bot Library](#) (Librería para realizar la conexión entre el Microcontrolador y el Bot de Telegram).
4. [WiFiClientSecure](#) (Comunicación segura a través del protocolo HTTPS).
5. [NTPClient](#) (Protocolo para sincronizar los relojes de los sistemas a través de redes de datos, permite que un microcontrolador se sincronice con un servidor de tiempo en Internet).
6. [ArduinoJson](#) (Librería de análisis JSON para Arduino).
7. [Explicación de funcionamientos de Relays.](#)
8. [Tutorial de referencia para utilizar una batería como motor de la mini bomba.](#)